

Architecture — Batch Meal Prep Planner

Companion to PRD — Batch Meal Prep Planner. Hand off both together. The PRD answers *what* to build and *why*; this document answers *how the system works internally*.

Architecture at a Glance

A plain-English summary of the four pillars and the data flow between them, before the technical detail.

Front End

Next.js 15 (App Router) · React · Tailwind CSS

This is everything Emma touches. Three screens rendered in the browser (Item Picker, Shopping List, Game Plan), built as React client components. Navigation is a persistent header. All UI state — which items are selected, the generated plan, the time window — lives in a React Context provider that wraps all three screens. There is no separate mobile app; the site is responsive and works on phone and laptop.

shadcn/ui is not currently specified in the PRD's constraints section (Tailwind only), so it is excluded for v1 unless explicitly added. Raise this before build begins if a component library is wanted.

Back End

Next.js 15 Server Action · Vercel (hosting)

The "back end" in this app is deliberately minimal — a single server action (`generateGamePlan`) inside the Next.js app. It is not a separate server or service; it runs as a Vercel serverless function. It exists for one reason only: to make the Anthropic API call securely without exposing the key to the browser. All other business logic (session estimate, shopping list aggregation, fallback sequencing) runs client-side in `/lib` functions. The server action returns typed JSON to the browser; it never writes to any store.

Database

None. localStorage only.

There is no database. The item library is a static JSON file bundled at build time (`/data/library.json`). The only persistence is the browser's `localStorage`, which stores the last set of selected item IDs under the key `lastPrep`. This resets if the user clears their browser data. No user data is ever sent to a server or stored anywhere outside the user's own browser.

External Services

Anthropic API (Claude) — one call per "See Game Plan" click.

This is the only external dependency at runtime. Claude receives the selected items and time window, and returns a sequenced JSON game plan. If the call fails or times out, the app falls back to a client-side rule-based sequencer — no external service is needed for the fallback.

Core Data Flow

The most important flow to understand is the Game Plan generation:

1. BROWSER

Emma selects items + time window → stored in React Context (memory only)

2. BROWSER → SERVER

Emma clicks "See Game Plan"

Browser calls the Next.js server action:

```
generateGamePlan(timeWindowMinutes, selectedItemIds)
```

3. SERVER (Vercel serverless function)

Receives item IDs; resolves full metadata from `library.json`; rejects unknown IDs

Constructs prompt per PRD §12

Calls Anthropic API with `ANTHROPIC_API_KEY` (never visible to browser)

Waits up to 12 seconds for response

4. ANTHROPIC API

Returns JSON: { steps: [...], totalHandsOnMinutes: number }

5. SERVER → BROWSER

Server action returns { ok: true, steps, totalHandsOnMinutes }

OR { ok: false } if timeout / invalid JSON (after one retry)

6. BROWSER

ok: true → renders step cards + hands-on time; saves plan to React Context

ok: false → client-side fallback sequencer runs; renders steps + fallback banner

On either path: localStorage["lastPrep"] = selected item IDs

The shopping list flow is entirely client-side — no server call, no API, no storage write:

React Context (selected items) → /lib/shopping.ts → aggregated + split list → rendered

The "Repeat last prep" flow:

App loads → read localStorage["lastPrep"] → cross-check IDs against library.json

→ if valid IDs found, show "Repeat last prep" button

→ on click, write IDs back to React Context

Binding Constraint

The `ANTHROPIC_API_KEY` must never reach the browser bundle. Every structural decision — where the Claude call lives, how state threads from client to server and back, how the fallback is positioned — flows from this single rule. A client-side Claude call would be two lines of code and zero architecture; the architecture exists solely to prevent it.

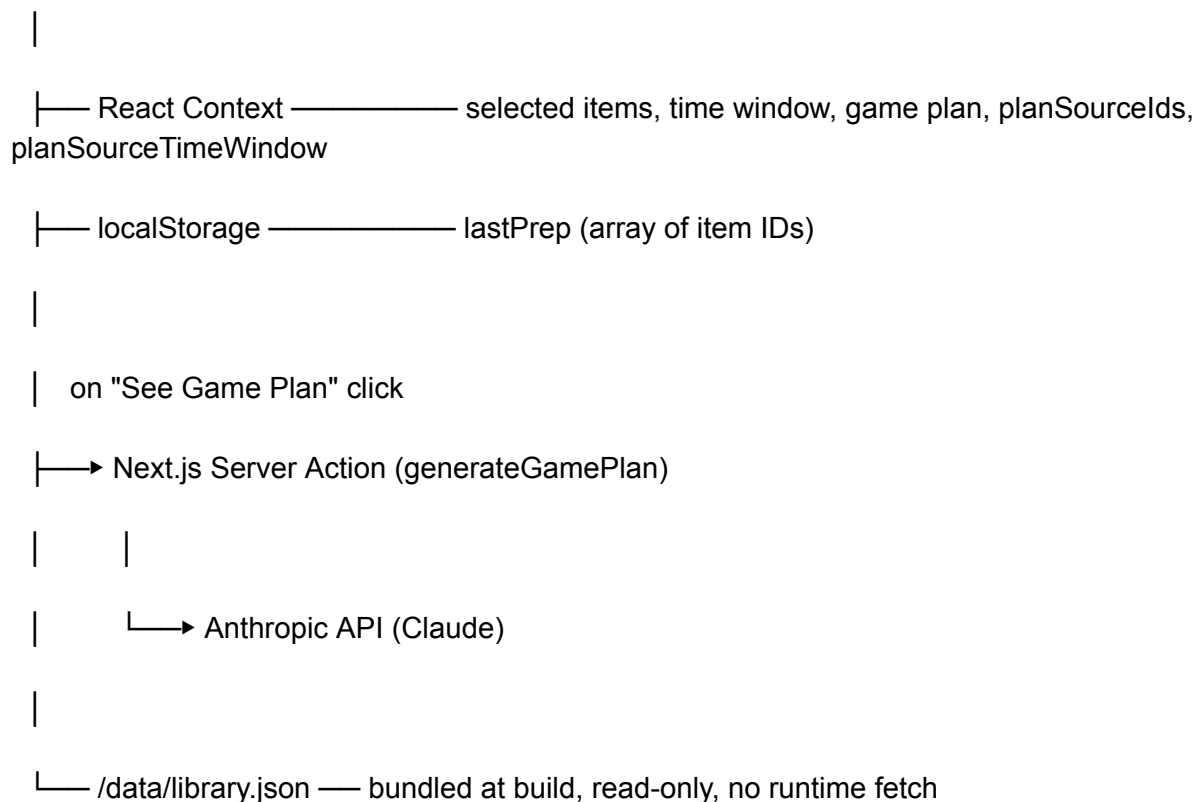
1. System Overview

Batch Meal Prep Planner is a stateless single-page web application served from Vercel. The item library is a static JSON file bundled at build time. All user state (selected items, generated

plan, time window) lives in a React context provider that resets on hard refresh. One server action crosses the trust boundary to call the Anthropic API; all other logic — shopping list aggregation, session estimate calculation, rule-based fallback sequencing — runs entirely in the browser. There is no database, no user auth, and no server-side persistence of any kind.

2. Topology

Browser (React)



Processes: One Next.js app on Vercel, deployed as independent serverless function instances. No shared in-process state between invocations. No background workers, no queues, no second service.

Data stores: One — browser `localStorage`. No database, no Redis, no S3.

External dependencies: One — Anthropic API (one call per "See Game Plan" click).

3. Module / Code Structure

/app

layout.tsx	# Root layout: mounts SelectionProvider, renders Header
page.tsx	# Screen 1 — Item Picker
/shopping-list	
page.tsx	# Screen 2 — Shopping List (client component, reads context)
/game-plan	
page.tsx	# Screen 3 — Game Plan (client component, calls server action)
/components	
Header.tsx	# Persistent nav: three links, disabled-state logic
ItemCard.tsx	# Single library item: name, category, time, select toggle
SessionEstimate.tsx	# Running estimate + overage warning + removal suggestion
ShoppingList.tsx	# Buy vs. Pantry split, aggregated ingredient list
GamePlanSteps.tsx	# Step cards + totalHandsOnMinutes display
StaleBanner.tsx	# "Selections changed — regenerate" banner
/context	
SelectionContext.tsx	# React context + provider: items, timeWindow, plan, planSourceIds
/lib	
types.ts	# TypeScript types for Item, Ingredient, GamePlan, Step
library.ts	# Import and type-assert library.json; export typed array
session.ts	# estimateSession(), detectOverage(), overageSuggestion()
shopping.ts	# aggregateIngredients(), splitBuyVsPantry()
sequencer.ts	# Rule-based fallback: sortItems(), assignSlots(), buildSteps()

game-plan.ts # 'use server' — generateGamePlan() — Anthropic call + fallback trigger

/data

library.json # Static item library (~30 items)

/scripts

validate-library.js # Pre-build validation; run via `npm run validate`

Module rules:

- `/app` page files are thin — they read context, render components, trigger the server action. No business logic.
- `/lib` is pure logic — no React imports, no context reads, no API calls, **except** `game-plan.ts` which is the sole server action.
- `game-plan.ts` is the **only** file that may import the Anthropic SDK. No other file touches it. It imports `server-only` at the top of the file — any accidental import from a non-server context will fail the build at compile time.
- `/components` may read `SelectionContext` but must not import `game-plan.ts` directly — the page triggers the server action and passes results down.
- `validate-library.js` is a plain Node script, not bundled into the app.

4. Data Architecture

Data	Store	Authority	Notes
Item library	<code>/data/library.json</code> (bundled)	Authoritative	Read-only. Shape enforced by <code>validate-library.js</code> at dev time and TypeScript at build time.
Selected item IDs	React context (memory)	Ephemeral	Lost on hard refresh. Restored from <code>lastPrep</code> if present on load.

Data	Store	Authority	Notes
Time window	React context (memory)	Ephemeral	Defaults to 90 min on load.
Generated game plan	React context (memory)	Derived	Regenerated on explicit user action only; never auto-updated.
Plan source item IDs	React context (memory)	Ephemeral	Stored alongside plan to power stale-plan detection. Compared against current selections on every toggle.
Plan source time window	React context (memory)	Ephemeral	Stored alongside <code>planSourceIds</code> . Stale detection compares both the item ID set and the time window against the plan's source values — if either differs, the stale banner shows.
Last prep item IDs	<code>localStorage["lastPrep"]</code>	Persistent (client)	Written on "Get Shopping List" or "See Game Plan" click — not on every selection change.

Data classification:

- *Personal*: nothing beyond item IDs — no PII, no health data, no user identity.
- *Public*: item library (no secrets in the JSON file).
- *Never server-side*: all user state. No user data ever leaves the browser except as part of the Anthropic request payload (item IDs and time window).

localStorage contract:

```
{ "lastPrep": ["protein-waffles", "energy-balls", "flank-steak-marinade"] }
```

If `localStorage` is unavailable (private browsing, storage quota exceeded), the "Repeat last prep" feature silently fails. All other functionality is unaffected.

5. Request & Event Flows

5a. Item selection + session estimate (client-only)

1. Emma sets the time window dropdown → `SelectionContext.setTimeWindow(n)`.
2. Emma toggles an item card → `SelectionContext.toggleItem(id)`.
3. `SessionEstimate` component reads context, calls `session.estimateSession(selectedItems)`, renders result + overage warning if applicable.
4. If overage: `session.overageSuggestion(selectedItems)` returns the item with highest `totalMinutes` (alphabetically last if tied) as the removal candidate.
5. `Header` reads context: if `selectedItems.length === 0`, Shopping List and Game Plan links render `aria-disabled` with `pointer-events-none`.

5b. Shopping list (client-only)

1. Emma navigates to `/shopping-list`. If context has zero items → `redirect('/')`.
2. `shopping.aggregateIngredients(selectedItems)` groups by name + unit, sums quantities.
3. `shopping.splitBuyVsPantry(aggregated)` splits on `isPantryStaple`; conflicting values default to `false` (Buy).
4. If any selected item has `isPantryHeavy: true` → single pantry-heavy banner rendered once.
5. On "Get Shopping List" click: `localStorage.setItem('lastPrep', JSON.stringify(selectedItemIds))`.

5c. AI game plan (client → server → Anthropic → client)

1. Emma clicks "See Game Plan" on `/game-plan`. If context has zero items → `redirect('/')`.
2. Component calls server action `generateGamePlan(timeWindowMinutes, selectedItemIds)`.
3. Loading state renders; button is disabled.
4. **Server action:** receives IDs, resolves full item metadata from `library.json` (rejects any unknown ID), constructs prompt per PRD §12, calls Anthropic with a 12-second timeout (first attempt).

5. **Happy path:** Response parsed as `{ steps, totalHandsOnMinutes }`. Returned to client as `{ ok: true, steps, totalHandsOnMinutes }`.
6. **Failure path:** If timeout → wait 2 seconds, retry with a 5-second timeout, return `{ ok: false, reason: 'timeout' }` on second failure. If non-200 → return `{ ok: false, reason: 'api_error' }` immediately (no retry). If invalid JSON → return `{ ok: false, reason: 'invalid_response' }` immediately. **Worst-case user wait before fallback: ~19 seconds** (12s first attempt + 2s wait + 5s retry), only on timeout.
7. Client receives result inside a `try/catch`:
 - `ok: true` → renders `GamePlanSteps`, saves plan + source item IDs + source time window to context.
 - `ok: false` → client calls `sequencer.buildFallback(selectedItems)`, renders steps with fallback banner: *"Using simplified plan — AI unavailable. Review steps before starting."*
 - `catch` (transport-level failure — network error, serialization error) → treated identically to `ok: false`; the client-side fallback runs.
8. `localStorage.setItem('lastPrep', ...)` written on this click.

5d. Stale plan detection (client-only)

1. On any item toggle or time window change after a plan exists: `SelectionContext` compares the current item ID set against `planSourceIds` **and** the current time window against `planSourceTimeWindow`.
2. If either differs → `StaleBanner` renders. Plan is not regenerated automatically.
3. Emma clicks "Regenerate" → flow 5c re-runs.

5e. Repeat last prep (client-only)

1. On app load: `SelectionContext` reads `localStorage["lastPrep"]`.
2. If present → "Repeat last prep" button renders in Item Picker.
3. On click → `SelectionContext.restorePrep(ids)` cross-references IDs against the loaded library (guards against stale IDs if the library is updated).

6. Interface Contracts

Server action: `generateGamePlan`

// /lib/game-plan.ts

'use server'

```

type GamePlanSuccess = {

  ok: true;

  steps: Step[];

  totalHandsOnMinutes: number;

}

type GamePlanFallback = {

  ok: false;

  reason: 'timeout' | 'api_error' | 'invalid_response';

}

export async function generateGamePlan(

  timeWindowMinutes: number,

  selectedItemIds: string[]

): Promise<GamePlanSuccess | GamePlanFallback>

```

The browser supplies only IDs — no metadata. The server action resolves the full `Item` objects from `library.json` and rejects any ID not found, so the browser has no way to inject fabricated metadata into the Anthropic prompt. The server action is the sole consumer of the Anthropic SDK.

Claude output shape

See PRD §12 (system prompt and output structure). Architecture adds: if the response body does not parse as JSON, or the parsed object does not match `{ steps: Step[], totalHandsOnMinutes: number }`, treat as a parse failure and enter the retry path.

localStorage schema

```
// key: "lastPrep"
```

```
type LastPrep = string[] // array of item IDs, e.g. ["protein-waffles", "energy-balls"]
```

TypeScript types — canonical, in [/lib/types.ts](#)

```
type Ingredient = {
```

```
  name: string;
```

```
  quantity: number;
```

```
  unit: string;
```

```
  isPantryStaple: boolean;
```

```
}
```

```
type Item = {
```

```
  id: string;
```

```
  name: string;
```

```
  category: 'breakfast' | 'snack' | 'protein' | 'sauce' | 'sides' | 'treat';
```

```
  handsonMinutes: number;
```

```
  totalMinutes: number;
```

```
  equipment: string[];
```

```
  ovenTemp: number | null;
```

```
  storage: ('fridge' | 'freezer' | 'pantry')[];
```

```
  isPantryHeavy: boolean;
```

```
  ingredients: Ingredient[];
```

```
}
```

```
type Step = {
```

```
  order: number;
```

```

    item: string;

    action: string;

    storage: 'fridge' | 'freezer' | 'pantry';

    note?: string;
  }

  type GamePlan = {

    steps: Step[];

    totalHandsOnMinutes: number;
  }

```

7. Cross-Cutting Patterns

API key

`ANTHROPIC_API_KEY` lives in Vercel environment variables. It is read inside the server action at call time. It is never imported in any client component, never passed to any client function. The `'use server'` directive ensures Next.js strips the module from client bundles. Next.js will throw a build error if a server-only import is used from a client component — this is the primary enforcement mechanism.

Error handling

The server action catches all exceptions and returns the typed union `GamePlanSuccess | GamePlanFallback`. The client wraps the server action call in a `try/catch`: the `try` block branches on `result.ok`; the `catch` block handles transport-level failures (network error, serialization error) and runs the same client-side fallback as an `ok: false` result. From the user's perspective, all failure modes produce the same fallback banner.

Redirects

`/shopping-list` and `/game-plan` check `selectedItems.length === 0` at render time. If zero → client-side redirect to `/`. Implemented via `useRouter().push('/')` in a `useEffect` (not a Next.js server redirect, since the zero-item state lives in client context).

Known UX rough edge: Because `useEffect` runs after the first render, the component will flash briefly with an empty/loading state before the redirect fires. This is expected behaviour, not a bug — do not attempt to suppress it by calling `router.push` synchronously during render (that is a side effect during render, which React forbids). If this becomes distracting in testing, the fix is to render a skeleton or `null` from the component while the effect runs.

No auth

There is no auth. This is not an oversight — single-user, no server state, nothing to protect. Do not add middleware or session logic.

8. State & Consistency

Transactional: Nothing. No shared mutable state.

Client-consistent: React context is synchronous. Session estimate, shopping list, and stale detection all compute from context on every render — no stale derived values.

The one async moment: Between "See Game Plan" click and Anthropic response, the plan is `null` and a loading state renders. There is no partial plan state.

Known races:

- Emma double-clicks "See Game Plan" → mitigated by disabling the button during loading (step 3 of §5c). Only one call can be in flight at a time.
-

9. Security Architecture

Trust boundaries:

- Browser ↔ Next.js server action: untrusted input. The server action validates inputs before use.
- Next.js server ↔ Anthropic API: trusted, HTTPS, API key auth.

API key protection:

- Stored in Vercel env vars, only accessible in server-side runtime.
- `'use server'` directive — Next.js strips the module from client bundles at build time.

- `game-plan.ts` imports the `server-only` package at the top of the file. This causes a hard build-time error if the module is ever imported from a client component or any file that could reach the client bundle — the idiomatic Next.js mechanism for enforcing server boundaries.

Input validation in the server action:

- `timeWindowMinutes`: must be a number in [30, 300], multiple of 30.
- `selectedItems`: each ID must exist in the static library. The server action resolves the full item from the library JSON — it does not pass browser-supplied name strings to the Anthropic prompt. This prevents prompt injection via fabricated item metadata.

Threat model:

Threat	Mitigation
API key exposed in client bundle	' <code>use server</code> ' + Next.js build-time stripping
Prompt injection via item name	Server resolves names from the library; browser cannot inject arbitrary strings into the prompt
Fabricated item IDs in server action payload	Server cross-references IDs against <code>library.json</code> ; unknown IDs are rejected
localStorage poisoning (fake item IDs in <code>lastPrep</code>)	<code>restorePrep()</code> filters IDs through the library — only valid IDs are restored
Public URL / API cost abuse	Live risk from deploy day. Accepted v1 risk (PRD §6) — this is a conscious decision, not a deferred one. Mitigation: add per-IP <code>middleware.ts</code> before sharing the URL publicly.

10. Failure Modes

Dependency	Failure	Handling	User sees	Alert?
Anthropic API	Timeout (>12s first attempt)	Wait 2s, retry with 5s timeout. If retry fails → {	Fallback plan + "Using simplified plan" banner	No (personal tool)

Dependency	Failure	Handling	User sees	Alert?
		<code>ok: false,</code> <code>reason:</code> <code>'timeout' }</code> . Worst-case wait: ~19s.		
Anthropic API	4xx / 5xx	Same as timeout	Same	No
Anthropic API	Invalid JSON response	Same as timeout	Same	No
<code>localStorage</code>	Unavailable	Try/catch on all read/write; silently skip	"Repeat last prep" button never appears; everything else works	No
<code>library.json</code>	Malformed / type mismatch	<code>validate-library.js</code> at dev time; TypeScript at build time	Build fails — never reaches production	N/A
<code>/shopping-list</code> or <code>/game-plan</code> accessed with zero items	No context state	Client redirect to <code>/</code>	Item Picker	No

Degraded modes:

- Anthropic down → rule-based fallback plan always available. App is fully functional without AI sequencing.
- `localStorage` unavailable → "Repeat last prep" silently absent. All three screens work normally.

11. Observability

v1 is a personal tool with no on-call, no SLOs, and no uptime requirements.

Logs: Vercel function logs capture server action invocations and unhandled errors. No custom logging needed.

Client errors: Browser console in development. No error reporting service.

Future hook: If API cost becomes a concern, log `usage.input_tokens + usage.output_tokens` from the Anthropic response inside the server action. Vercel function logs capture anything written to `console.log` in a server action.

12. Deployment & Environments

Environments:

- `local` — `next dev`. Uses `.env.local` for `ANTHROPIC_API_KEY`. Real API key required for game plan testing (no mock).
- `prod` — Vercel. `ANTHROPIC_API_KEY` set in Vercel project dashboard.

No staging environment. Personal tool with no pre-prod verification requirement.

Pipeline: Push to `main` → Vercel auto-deploys. No separate CI pipeline for v1. Library validation and TypeScript checking run as part of the Vercel build via `"build": "npm run validate && next build"` in `package.json` — a malformed `library.json` or type error will fail the build before reaching production.

Library updates: Edit `library.json`, run `npm run validate`, push. No migrations. No deployment coordination.

Before sharing the URL publicly: Add per-IP rate limiting via `middleware.ts` (see §14 Non-Goals and §9 threat model). This is a named pre-launch checklist item — the deployed app is live from the first push and the Anthropic API key is billed per call.

Rollback: Vercel instant rollback to any previous deployment via dashboard (≈1 minute). Library changes are git-reverted.

13. Architecture Decision Records (ADRs)

#	Decision	Rationale	Rejected alternatives
001	Next.js server action for Claude call, not a separate <code>/api</code> route	Colocated with the triggering page; no public URL created; simpler end-to-end typing	<code>/api/game-plan</code> route (creates an undocumented public endpoint; extra file)
002	React Context for selection state	No extra dependency; state doesn't need to survive hard refresh (localStorage handles that); three screens share one provider cleanly	URL search params (too long for 30 items), Zustand (extra dep for simple use case)
003	<code>library.json</code> bundled at build time, not fetched at runtime	Zero latency; works offline; zero infrastructure; fits the "no database" hard constraint	Supabase table (overkill), runtime API route (pointless network call for static data)
004	Rule-based fallback runs client-side, not as a second server call	If Anthropic is down, the fallback must not add a second network dependency. Client-side fallback is always available.	Server-side fallback route (slower; defeats purpose of a degraded-mode fallback)
005	Server action with <code>'use server'</code> , not client-side Anthropic SDK	Direct consequence of the binding constraint — API key protection	Client-side SDK call (impossible; exposes key in browser bundle)
006	Single JSON response, no streaming	Simpler: structured JSON is easy to validate and type; streaming adds	Streaming server action (progressive render but harder to validate schema)

#	Decision	Rationale	Rejected alternatives
		parsing complexity for minimal UX benefit in a planning tool. Trade-off: users may wait up to ~19 seconds (12s first attempt + 2s wait + 5s retry) with no progressive feedback before the fallback renders — accepted because meal prep plans are short and structured JSON validation is the priority.	
007	Server action resolves item metadata from library, not from browser payload	Prevents prompt injection — the browser cannot supply arbitrary strings into the Anthropic prompt	Trust browser-supplied item names (simpler, but allows prompt injection)

14. Architectural Non-Goals

- **No server-side session or auth.** Single-user; nothing to protect server-side.
- **No database.** Static JSON is the library. No ORM, no migrations, no connection pooling.
- **No background workers or queues.** One synchronous server action is the entire async surface.
- **No streaming.** Game plan returns as a single validated JSON payload.
- **No rate limiting for v1.** Live risk from deploy day — accepted consciously per PRD §6. Add per-IP `middleware.ts` before sharing the URL publicly.
- **No staging environment.** Push to `main` → prod. Personal tool.
- **No SSR of dynamic screens.** Shopping List and Game Plan are client-rendered from React context; Next.js SSR cannot access context or `localStorage`.
- **No error monitoring service.** Vercel function logs are sufficient.

- **No unit test suite for v1.** `validate-library.js` and manual smoke testing are the test strategy. Add tests when the fallback sequencer logic stabilises.
 - **No CDN for `library.json`.** It is bundled — no runtime fetch, no cache invalidation needed.
-

15. Future Evolution Hooks

- **Custom recipe extraction:** Add a Route Handler (`/app/api/extract/route.ts`) — not a server action — that accepts a URL or PDF, calls Claude to extract item metadata, validates against the §6 schema, and returns a candidate item for user confirmation. Because this feature may need to be callable from outside the app (e.g., a browser extension), it needs a public URL, which means it requires its own auth and rate-limiting strategy and falls outside ADR 001's reasoning. The library JSON schema already supports the data shape — no structural change needed.
- **Cross-device sync / accounts:** Replace `localStorage["lastPrep"]` reads with a `GET /api/user/last-prep` call, and writes with a `POST`. React context shape is unchanged — only the persistence layer swaps.
- **Streaming game plan:** The server action's return type changes to a `ReadableStream`; `GamePlanSteps` renders progressively as chunks arrive. JSON validation moves to a stream-completion handler. No other structural change.
- **Rate limiting:** Add `middleware.ts` at the app root, checking a Redis (or KV) counter per IP, wrapping the server action route. One file addition, no architectural change to existing code.
- **Swapping Claude:** `/lib/game-plan.ts` is the only file that touches the Anthropic SDK. Swap the SDK call behind the same `generateGamePlan` function signature — nothing else changes.
- **Repeat last prep across devices:** Today stored in `localStorage`. Migrating to a server-side user record (with auth) requires only changing the two read/write calls in `SelectionContext` — the shape of `lastPrep` (array of item IDs) is already the right portable format.